

Identifying and Correcting Level Design Bottlenecks via Deep Reinforcement Learning

Robert Ronald Calla Maguiña
Department of Informatics, PUC-Rio
Rio de Janeiro, RJ, Brazil

Abstract

This work reports on the progress of an Artificial Intelligence system aimed at automatically identifying and correcting *level design* bottlenecks in physics-based 2D stages. The development environment is built on the Godot 4.7 engine, targeting multiplayer mechanics. Unlike traditional navigation (*pathfinding*) approaches, the goal is to train an agent capable of analyzing the topology of a map, identifying congestion zones that hinder gameplay flow, and applying content-generation techniques to suggest scenario adaptations. We propose an architecture that combines a structured 2D-grid representation of the environment, spatial feature extraction via a Convolutional Neural Network (CNN), and a Deep Reinforcement Learning (DRL) agent inspired by *Curiosity-Driven Exploration* and *Procedural Content Generation via Reinforcement Learning* (PCGRL). We formalize the problem as a Markov Decision Process, specify a concrete network architecture (convolutional feature extractor, actor-critic heads, and Intrinsic Curiosity Module) together with a reference set of PPO hyperparameters, describe the current implementation status, and lay out an evaluation methodology – with explicitly defined metrics, baselines, and an experimental protocol – for the next development cycles. As of this writing, agent training has not yet been executed; no performance results are reported, and every number in Section 4 is a proposed design choice to be tuned empirically, not an outcome of a completed experiment. We also discuss the main methodological risks, most notably *reward hacking*, and outline a roadmap of future work.

Keywords

Level Design, Deep Reinforcement Learning, Procedural Content Generation, Curiosity-Driven Exploration, Godot Engine, Automated Playtesting, Convolutional Neural Networks, Proximal Policy Optimization

1 Introduction

Level design is one of the pillars of the player experience: the way corridors, rooms, obstacles, and alternative routes are arranged directly determines the pacing, the perceived difficulty, and the fluidity with which a player progresses through a stage. However, manually identifying friction points that hurt this fluidity – so-called *bottlenecks* – is a costly, subjective process that depends on extensive human *playtesting* sessions. In productions with short iteration cycles, or in procedurally generated stages, this cost becomes even more critical, since every new version of a level would require a new round of manual testing.

This work reports on the progress of research and development of an Artificial Intelligence application for identifying and correcting these bottlenecks. The development environment is being built on the Godot 4.7 [2] engine, focusing on physics-based 2D stages and multiplayer mechanics – a domain in which navigation bottlenecks affect not just a single player, but the dynamics of multiple agents competing for the same space.

1.1 Motivation

The central motivation of this project is that automated, AI-based playtesting tools can reduce the iteration cost of the level-design process, allowing designers to receive quantitative feedback about a level’s structural quality before ever exposing it to human testers. Unlike static graph-analysis heuristics (for example, centrality measures over the navigability graph), a reinforcement-learning agent can capture dynamic gameplay effects – such as congestion caused by multiple players competing for the same corridor – that are not trivial to express as a fixed rule.

1.2 Problem Statement

The goal of the project is to go beyond simple navigation (*pathfinding*). The aim is to train a machine-learning agent capable of:

- (1) analyzing the topology of the map from a structured representation that is independent of graphical rendering;
- (2) identifying congestion zones that hurt gameplay flow, i.e., regions of the map where the traversal flow between points of interest is abnormally low; and
- (3) applying content-generation techniques to suggest scenario adaptations that alleviate these bottlenecks, closing the loop between diagnosis and correction.

In other words, the agent should not merely traverse the level, but learn to point out – and eventually correct – structural design flaws.

1.3 Contributions

The contributions of this progress report are:

- an environment-representation architecture that is independent of graphical rendering, based on 2D matrices extracted directly from Godot’s `TileMap` and `NavigationServer2D`;
- a formalization of the problem as a Markov Decision Process (MDP), with a state space, an action space,

and a proposed hybrid (intrinsic + extrinsic) reward function;

- a concrete, fully specified network architecture – convolutional feature extractor, actor-critic heads, and Intrinsic Curiosity Module – together with a reference set of PPO hyperparameters, proposed as a starting point for the upcoming implementation;
- a planned evaluation methodology, with explicitly defined metrics (including their formulas), baselines, and an experimental protocol, still to be executed in the next project cycles; and
- an initial discussion of methodological limitations and risks, most notably the risk of *reward hacking*.

1.4 Report Organization

The remainder of this article is organized as follows. Section 2 discusses related work in procedural content generation via reinforcement learning, automated playtesting, and curiosity-driven exploration. Section 3 formalizes the problem as an MDP. Section 4 details the proposed system architecture, including the network design and hyperparameters. Section 5 describes the current implementation status. Section 6 proposes the evaluation methodology. Section 7 discusses limitations and risks. Section 8 discusses ethical considerations and responsible use. Section 9 presents the future-work roadmap, and Section 10 concludes the article.

2 Related Work

The use of reinforcement learning for game-content generation and evaluation has been consolidating as an alternative to traditional rule-based or search-based procedural content generation (PCG) [8]. This section reviews three lines of work that underpin the proposed architecture: procedural content generation via reinforcement learning, automated playtesting with human-like agents, and curiosity-driven exploration.

2.1 Procedural Content Generation via Reinforcement Learning

Khalifa et al. [3] propose PCGRL (*Procedural Content Generation via Reinforcement Learning*), in which level generation is formulated as a sequential decision-making problem: an agent edits the map tile by tile and receives rewards according to quality metrics of the resulting level (for example, connectivity, number of reachable enemies, or shortest-path length). Unlike generative models trained through supervised learning on corpora of existing levels, PCGRL does not depend on large annotated datasets, but on a well-defined reward function over the space of levels – a central characteristic that we aim to reuse in this project. This formulation directly inspires the state representation and action space discussed in Section 4 of this work.

2.2 Automated Playtesting with Human-Like Agents

On the automated-playtesting front, De Woillemont et al. [1] use reinforcement learning to generate agents with human-like play styles, enabling a game to be tested under varied behavior profiles without relying on human testers for every iteration. This line of work starts from the premise that a purely optimal agent (for example, one that always finds the shortest path) is not representative of real player behavior, since players explore, hesitate, and make mistakes. Sestini et al. [7] extend this line by proposing curiosity-conditioned proximal trajectories for automated *gameplay* validation, combining curiosity-guided exploration with trust-region policy-optimization methods to detect rare or problematic game states (for example, *softlock* situations or bug exploitation).

2.3 Curiosity-Driven Exploration

The curiosity-guided exploration component used in this project follows the formulation of the Intrinsic Curiosity Module (ICM) proposed by Pathak et al. [5], in which a predictive model learns to anticipate the latent representation of the next state from the current state and action; the prediction error is used as an intrinsic reward, encouraging the agent to visit regions of the environment whose dynamics have not yet been well modeled. This mechanism is particularly well suited to our scenario, since level-design bottlenecks tend to correspond to regions that are seldom visited by naive navigation policies – exactly the regions that a curiosity signal tends to prioritize.

2.4 Positioning of This Work

Table 1 summarizes how the present work relates to the three works discussed above. This project sits at the intersection of these lines: like PCGRL, it uses a matrix-based level representation as the state of a Markov decision process; and, like the automated-playtesting works, it incorporates curiosity-guided exploration – not to generate player styles, but so that the agent itself discovers and flags regions of the map with low traversal flow, treating bottleneck detection as the central objective of the reward, rather than merely a by-product of exploration.

3 Problem Formalization

3.1 Level Design Bottlenecks

We informally define a bottleneck as a region of the level whose traversal flow – the number of times agents pass through that region during a gameplay episode, normalized by the total episode duration – is significantly lower than expected for a region with equivalent connectivity in the navigation graph. This definition captures both “structural” bottlenecks (a narrow corridor that physically limits the simultaneous passage of multiple players) and “behavioral” bottlenecks (a fork that players tend to avoid because it subjectively seems less advantageous).

Table 1: Positioning relative to related work

Work	Main focus
PCGRL [3]	Tile-by-tile level generation via RL, reward over global level-quality metrics.
De Woillemont et al. [1]	Generation of playtesting agents with human-like play styles.
Sestini et al. [7]	Automated gameplay validation via curiosity-conditioned exploration.
This work	Diagnosis and flagging of level-design bottlenecks as the central reward objective, combining matrix representation (PCGRL) with curiosity (ICM) to prioritize low-flow regions.

3.2 Markov Decision Process

We model the problem as a Markov Decision Process (MDP) $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, in which:

- $\mathcal{S}$ is the state space, defined by the 2D matrix $G \in \{0, 1\}^{H \times W}$ that represents the level (Section 3.3), possibly augmented with additional channels encoding the agent’s position and points of interest (entrance, exit, items);
- $\mathcal{A}$ is the action space, detailed in Section 3.4;
- $P(s_{t+1} \mid s_t, a_t)$ is the transition dynamics, determined by Godot’s physics and navigation engine;
- $R(s_t, a_t, s_{t+1})$ is the reward function, detailed in Section 3.5; and
- $\gamma \in [0, 1]$ is the discount factor.

The training objective is to find a policy $\pi_\theta(a_t \mid s_t)$, parameterized by a neural network with parameters θ , that maximizes the expected return

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t, s_{t+1}) \right]. \quad (1)$$

3.3 State Space

The state observed by the agent at each instant t is the matrix G_t described in Section 4.2, processed by a convolutional feature extractor $\phi(\cdot)$ (Section 4.3) that produces a latent feature vector $z_t = \phi(G_t) \in \mathbb{R}^d$. This latent vector is the input to both the policy network and the curiosity module.

3.4 Action Space

We propose a hybrid action space, reflecting the two-phase nature of the problem (navigation and, later, content editing):

- **Navigation actions:** a discrete displacement vector $a^{\text{nav}} \in \{\text{north, south, east, west, stay}\}$, matching the granularity of the tile matrix;
- **Flagging/editing actions** (later project phase): $a^{\text{edit}} \in \{\text{none, flag bottleneck, remove blockage, add blockage}\}$ applied to the matrix cell corresponding to the agent’s current position.

In the current project phase, only a^{nav} is in use; the editing space will be incorporated progressively, as discussed in Section 9.

3.5 Reward Function

We propose a hybrid reward function, combining an extrinsic term r_t^{ext} , responsible for penalizing bottlenecks, and an intrinsic term r_t^{int} , responsible for guiding exploration:

$$R(s_t, a_t, s_{t+1}) = r_t^{\text{ext}} + \beta r_t^{\text{int}}, \quad (2)$$

where β is a coefficient balancing the two terms.

The extrinsic term penalizes cells with low traversal flow. Let $\tau(v)$ be the accumulated traversal flow of cell v over a window of N episodes, and θ_τ a threshold below which a cell is considered a bottleneck. We define

$$r_t^{\text{ext}} = - \sum_{v \in \mathcal{V}(s_t)} w(v) \cdot \mathbb{I}[\tau(v) < \theta_\tau], \quad (3)$$

where $\mathcal{V}(s_t)$ is the immediate neighborhood of the agent’s position in s_t and $w(v)$ is a weight proportional to the centrality of cell v in the navigation graph – so that bottlenecks on central routes are penalized more strongly than bottlenecks in marginal regions of the map.

The intrinsic term follows the ICM formulation [5]: a predictive model f_ψ estimates the latent representation of the next state, $\hat{z}_{t+1} = f_\psi(z_t, a_t)$, and the prediction error is used as a reward,

$$r_t^{\text{int}} = \frac{\eta}{2} \|\hat{z}_{t+1} - z_{t+1}\|_2^2, \quad (4)$$

where η is a scaling constant. The intuition is that regions of the level whose dynamics are poorly modeled by the agent – typically seldom-visited regions, such as bottlenecks – produce a larger prediction error and therefore a larger intrinsic reward, complementing the extrinsic signal of Equation (3) during the early stages of training, when the estimates of $\tau(v)$ are still noisy.

3.6 Illustrative Example

To make the formalization above concrete, consider the hypothetical corridor shown in Figure 1: a 5×3 matrix in which the center cell of the middle column is the only passage point between the left and right halves of the level.

In this example, a naive navigation agent (for example, one trained only with a sparse reward for reaching the exit) tends to cross cell **B** with the same frequency as any other cell along the path, since it is, by definition, the only possible route – there is therefore no signal that this cell is structurally different from the others simply by observing visitation frequency. This is why Equation (3) weights the flow $\tau(v)$ by the centrality $w(v)$ of the cell in the navigation graph: cell **B** has a much higher betweenness centrality than the other cells in the corridor, since every path between the two halves of the level necessarily passes through it. A value of $\tau(v)$ comparable to that of the other free cells, combined with a high $w(v)$, is what flags **B** as a relevant structural bottleneck, rather than just another passage cell. In corridors with multiple alternative paths, on the other hand, the centrality

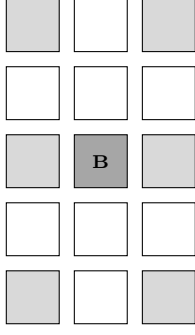


Figure 1: Illustrative example of a structural bottleneck: light-gray cells are walls, white cells are free path, and the dark-gray cell marked “B” is the only passage point between the two halves of the level.

of each individual cell tends to be lower, and a bottleneck only emerges when the observed flow $\tau(v)$ falls significantly below what is expected for cells of comparable centrality – it is this combination of static topology ($w(v)$) and observed dynamic behavior ($\tau(v)$) that we aim to capture with the proposed reward function, and that a purely structural heuristic (considering only $w(v)$) could not distinguish on its own.

4 System Architecture

Figure 2 illustrates the proposed pipeline, from the Godot environment to the action policy, passing through the components described below. Unless stated otherwise, every architectural and hyperparameter choice in this section is a *proposed* design decision for the upcoming implementation, not a value obtained from a completed experiment.

4.1 Development Environment

The development environment is being built on the Godot 4.7 [2] engine, chosen for its native support for deterministic 2D physics, its navigation system (`NavigationServer2D`), and the ease of introspecting scene data via GDScript, which allows the structured level representation to be extracted without any image-processing step.

4.2 Environment Representation

After evaluating the best input strategy for the AI, we discarded the use of direct camera images (pixel rendering): processing textures and lighting would add unnecessary computational cost to the neural network, without contributing information relevant to the bottleneck-identification task, which is fundamentally structural, not visual.

The established solution is the structured extraction of the map as a 2D matrix (*grid*). Using Godot 4.7’s built-in features, the `TileMap` and the `NavigationServer2D` meshes are mapped to produce a two-dimensional array whose values represent terrain states:

- 0: free, navigable path;
- 1: wall or rigid obstacle.

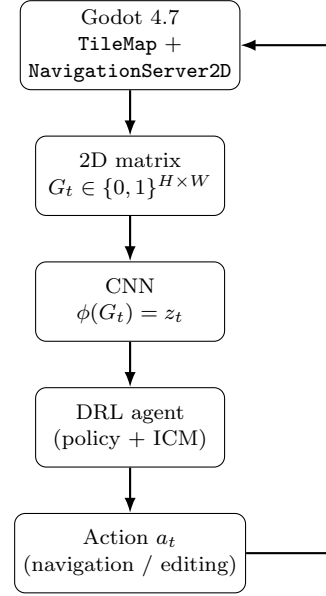


Figure 2: Proposed pipeline: extraction of the level matrix, spatial processing via CNN, the DRL agent’s decision, and feedback of the action to the Godot environment.

At this stage of the architectural project, the generation of this clean matrix by Godot has been isolated as an already-functional “black box”, allowing full attention to be devoted to modeling the AI. The initial binary encoding ($\{0,1\}$) was chosen for its simplicity; additional channels (agent position, distance to exit, historical traffic density) can later be concatenated as extra planes of the same matrix without changing the convolutional extractor’s architecture, only the number of input channels.

4.3 Convolutional Feature Extractor: Proposed Architecture

The generated 2D matrix acts as the observation (*state*) for the neural network. To extract the topological patterns of this spatial environment – such as narrow corridors, critical routes, and obstacle density – the data is proposed to pass through a Deep Convolutional Network [4] (CNN). The nature of CNNs allows the model to understand the level geometrically, rather than merely reading isolated numbers, which is essential for recognizing the spatial patterns that characterize a bottleneck.

Table 2 specifies the proposed feature-extractor architecture, following the pattern established by grid-world RL work: small 3×3 kernels with stride 1, preserving the tile granularity needed to precisely localize a bottleneck. We deliberately avoid aggressive max-pooling operations, which would tend to blur exactly the fine structures (corridors one tile wide) that the system needs to detect.

Table 2: Proposed convolutional feature extractor $\phi(\cdot)$. Input: $G_t \in \{0, 1\}^{C \times H \times W}$, with $C = 1$ initially.

Layer	Filters/Units	Kernel	Stride	Act.
Conv1	32	3×3	1	ReLU
Conv2	64	3×3	1	ReLU
Conv3	64	3×3	1	ReLU
Flatten	—	—	—	—
FC1 (features)	256	—	—	ReLU

The output of FC1 is the latent feature vector $z_t \in \mathbb{R}^{256}$ ($d = 256$), shared between the policy/value networks and the curiosity module described below.

4.4 Policy and Value Networks

We propose an actor-critic architecture with a shared convolutional trunk $\phi(\cdot)$ (Table 2) and two separate heads:

- **Actor head:** a fully connected layer mapping $z_t \in \mathbb{R}^{256}$ to $|\mathcal{A}|$ logits, followed by a softmax over the discrete action set ($|\mathcal{A}^{\text{nav}}| = 5$ in the current navigation-only phase; extended to $|\mathcal{A}^{\text{nav}}| + |\mathcal{A}^{\text{edit}}| = 9$ once editing actions are added, per Section 3.4);
- **Critic head:** a fully connected layer mapping z_t to a single scalar, the state-value estimate $V_\omega(s_t)$, used to compute the advantage estimates required by PPO.

4.5 Intrinsic Curiosity Module: Proposed Architecture

Following Pathak et al. [5], the ICM reuses the feature encoder $\phi(\cdot)$ and adds two small networks:

- **Inverse model:** takes the concatenation of z_t and z_{t+1} (512 units), passes it through one fully connected hidden layer of 256 units (ReLU), and predicts the action a_t that produced the transition, via a softmax output over $\mathcal{A}^{\text{nav}}$. It is trained with a cross-entropy loss L_I , and its role is to shape $\phi(\cdot)$ so that z_t only encodes features relevant to the agent’s own actions, ignoring aspects of the environment the agent cannot control;
- **Forward model (f_ψ):** takes the concatenation of z_t and a one-hot encoding of a_t ($256 + |\mathcal{A}^{\text{nav}}|$ units), passes it through one fully connected hidden layer of 256 units (ReLU), and predicts $\hat{z}_{t+1}$. It is trained by minimizing the mean-squared error between $\hat{z}_{t+1}$ and z_{t+1} , which is exactly the quantity used as the intrinsic reward in Equation (4).

The combined ICM training loss follows the standard weighting proposed by Pathak et al.:

$$L_{\text{ICM}} = (1 - \lambda) L_I + \lambda L_F, \quad (5)$$

with $\lambda = 0.8$ as the literature-informed reference value, giving more weight to the forward-model loss L_F (which directly produces the intrinsic reward) while still using the inverse-model loss L_I to keep the learned features action-relevant.

Table 3: Components of the Deep Reinforcement Learning system

Component	Strategy in the project
Observation (<i>State</i>)	The 2D matrix processed by the CNN, representing the stage’s instantaneous topology ($z_t = \phi(G_t)$).
Action	Discrete vectorial navigation and, later, the ability to flag or modify tiles (add/remove blockages).
Reward	Extrinsic/intrinsic combination: strongly penalizes low traversal-flow areas (bottlenecks) and rewards exploration of regions poorly modeled by the agent.

4.6 Deep Reinforcement Learning Agent and Proposed Hyperparameters

The agent’s decision-making and map exploration rely on a Deep Reinforcement Learning architecture, combining Curiosity-Driven Exploration [5] with the foundations of PCGRL [3]. Table 3 summarizes the three central components of the system per the formalization presented in Section 3.

Regarding the policy-optimization algorithm, we opt to use *Proximal Policy Optimization* (PPO) [6], rather than value-based methods such as Deep Q-Networks. This choice is justified by three reasons: (i) the action space proposed in Section 3.4 is naturally discrete, but will be extended to continuous edit-intensity variations in later phases, and PPO accommodates both cases without a structural change; (ii) PPO is known for its training stability in scenarios with sparse and noisy rewards – exactly the scenario expected when combining the extrinsic and intrinsic terms of Equation (2); and (iii) its on-policy nature simplifies integration with the curiosity module, which depends on recent trajectories that are consistent with the current policy so that the dynamics-model prediction error remains informative.

Table 4 lists the reference PPO hyperparameter configuration we propose as a starting point for implementation, following common defaults reported by Schulman et al. [6] and widely adopted in standard PPO implementations. These values are explicitly a proposed initial configuration, to be empirically tuned once training begins (see Section 9); they are not the result of a hyperparameter search.

The wide range proposed for β reflects a known open calibration issue also flagged in Section 7: the relative scale of the intrinsic and extrinsic reward terms is environment-dependent and is expected to require empirical adjustment, rather than being fixed a priori.

4.7 Data Integration and Training Pipeline

Integration between the Godot environment and the training environment (Python) is planned via a local-socket communication protocol, in which Godot exports, at every simulation *step*, the serialized matrix G_t and receives back the action

Table 4: Proposed reference PPO hyperparameters (starting configuration, not yet tuned)

Hyperparameter	Proposed value
Optimizer	Adam
Learning rate	3×10^{-4}
Discount factor γ	0.99
GAE λ	0.95
Clipping range ε	0.2
Rollout length (steps/env)	2048
Minibatch size	64
Epochs per update	10
Entropy coefficient	0.01
Value-loss coefficient	0.5
Max gradient norm	0.5
Curiosity weight β (Eq. (2))	0.01–0.1 (to be calibrated)
Forward-loss scale η (Eq. (4))	1.0
ICM loss weight λ (Eq. (5))	0.8

a_t chosen by the policy. This process separation allows training to be accelerated by dedicated hardware (GPU) without overloading the game engine’s rendering and physics loop, and will later allow multiple environment instances to be parallelized to speed up trajectory collection, a common practice in deep RL pipelines.

5 Current Implementation Status

With the base communication layer and the AI scope (2D matrix $\rightarrow$ CNN $\rightarrow$ DRL) validated and defined, the project is moving toward the formal structuring of the machine-learning components. As of this writing:

- the extraction of the 2D matrix from the **TileMap** and the **NavigationServer2D** has been implemented and isolated as an independent module;
- **preliminary validation performed:** for the handful of test levels used during development, the extracted matrix was manually inspected and confirmed to match the corresponding **TileMap** layout cell-by-cell; this is a functional/qualitative sanity check of the extraction module, not a performance evaluation of the agent, since no agent has been trained yet;
- the overall design of the CNN + DRL pipeline (Section 4), including the concrete network architecture and the reference PPO hyperparameters, has been defined and cross-checked against the PCGRL and curiosity-driven-exploration literature;
- the formalization of the problem as an MDP (Section 3), including the initial hybrid reward function proposal, was completed at this stage of the report; and
- **agent training has not yet started:** no PPO run, no learning curve, and no bottleneck-detection accuracy number exists at the time of writing. The items above constitute the architectural groundwork

on which the experiments described in Section 6 will be conducted.

6 Proposed Evaluation Methodology

Since agent training is still in the design stage, this section describes the evaluation methodology planned for the next project cycles, and **not** experimental results already obtained. No test run of the full agent has been performed; the metrics, baselines, and protocol below define what will be measured once training begins, not what has been measured so far.

6.1 Metrics

We propose the following metrics, with explicit formulas, to evaluate the agent’s ability to identify bottlenecks. Let $\hat{B}$ be the set of cells flagged by the agent as bottlenecks and B^* the set of cells identified as bottlenecks by human experts on a manually annotated reference-level set. Define true positives $TP = |\hat{B} \cap B^*|$, false positives $FP = |\hat{B} \setminus B^*|$, and false negatives $FN = |B^* \setminus \hat{B}|$.

- **Detection precision:**

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (6)$$

- **Coverage (recall):**

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (7)$$

- F_1 **score**, summarizing precision and recall in a single number:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (8)$$

- **Visitation-map entropy:** let $p(v) = \text{visits}(v) / \sum_u \text{visits}(u)$ be the empirical visitation distribution over grid cells during an episode. The Shannon entropy

$$H = - \sum_v p(v) \log p(v) \quad (9)$$

is used as a proxy to check whether curiosity-guided exploration is indeed covering low-flow regions, rather than merely reinforcing already well-known routes: a higher H indicates a more spatially dispersed visitation pattern.

- **Post-editing bottleneck reduction:** for the future editing-action phase (Section 3.4), the relative change in traversal flow $\tau(v)$ before and after the agent’s suggested interventions,

$$\Delta\tau(v) = \frac{\tau_{\text{after}}(v) - \tau_{\text{before}}(v)}{\tau_{\text{before}}(v)}, \quad (10)$$

with $\Delta\tau(v) > 0$ indicating that the intervention increased traversal flow through a previously identified bottleneck cell.

6.2 Baselines

As a comparison reference, we plan to include:

- a random agent, executing navigation actions uniformly at random, to calibrate the performance floor for the metrics above;
- a classical *pathfinding* agent (A* search over the navigation graph), representing the “no-learning” baseline and allowing the specific gain attributable to the DRL component to be isolated; and
- a variant of the proposed agent itself without the intrinsic curiosity term ($\beta = 0$ in Equation (2)), to measure the isolated contribution of curiosity-guided exploration relative to a purely extrinsic reward.

6.3 Experimental Protocol

The planned protocol consists of training each agent configuration on a fixed set of training levels, evaluating periodically on a validation set of levels not seen during training, in order to verify the policy’s generalization to topologies different from those used in training – an essential requirement for the tool to be useful in real level-design scenarios, where the agent must evaluate novel levels, not only the levels it was trained on. Each configuration will be run with multiple random seeds to allow variance analysis across runs, a practice recommended in the reinforcement-learning literature [9] given the well-known sensitivity of these methods to initialization and to the sampling order of trajectories.

7 Discussion: Limitations and Risks

Although the proposed architecture is promising, some methodological risks deserve explicit discussion at this stage of the project.

Risk of reward hacking. Combining extrinsic and intrinsic rewards (Equation (2)) introduces the classic risk that the agent exploits artifacts of the reward function itself instead of solving the intended problem – for example, lingering in regions whose dynamics model is intentionally hard to predict (such as areas with chaotic physics), producing high intrinsic reward without corresponding to an actual design bottleneck. Mitigating this risk will require careful normalization of the r_t^{int} term and, possibly, its decay over the course of training, as the dynamics model becomes more accurate.

Generalization across visual themes and topologies. Since the state representation is purely structural (a binary matrix), better generalization across distinct visual themes is expected than for pixel-based approaches; however, generalization across topologies that are structurally very different from those seen in training (for example, levels with non-orthogonal geometry) has not yet been evaluated and remains an open risk.

Computational cost of distributed training. The separation between the Godot process and the training process (Section 4) introduces communication overhead that could become a performance bottleneck itself – somewhat ironically, the training pipeline may suffer from the very type of problem the agent is being trained to detect at the level of the game. This overhead will need to be measured empirically before scaling up the number of parallel environments.

Validation with human experts. The precision and recall metrics proposed in Section 6 depend on manual bottleneck annotation by experts, a process that introduces subjectivity and still needs to be operationalized – for example, by defining an annotation protocol with multiple annotators and inter-annotator agreement measures.

8 Ethical Considerations and Responsible Use

Automated level-design diagnostic tools raise responsible-use questions that deserve explicit consideration, even at this early stage of the project.

Complementing, not replacing, human judgment.

The proposed system is meant to provide an additional quantitative signal to the design process, not to replace human designers’ judgment of what makes a level interesting. A bottleneck flagged by the agent may, in certain intentional design contexts (for example, a tactical choke point in a competitive map), be a desired feature rather than a defect to be corrected. The tool should therefore be used as decision support, with the final call remaining with the design team.

Bias in the definition of fluidity. The traversal-flow metric $\tau(v)$ and the centrality weights $w(v)$ (Section 3.5) embed a specific – and necessarily partial – definition of what constitutes a “fluid” level. This definition may not be representative of every play style or player profile (for example, players who prefer slow exploration over fast progression), and should be communicated transparently to whoever uses the tool, so that automated recommendations are not interpreted as objective truths about design quality.

Real player data. At the current project stage, all training happens in simulation, with no collection of human player data. Should future stages incorporate real playtesting data – for example, to calibrate $\tau(v)$ with telemetry from human matches – an explicit consent and anonymization protocol will be required, following standard practices for research with human participants.

9 Future Work

With the architecture and problem formalization defined, the project’s next steps include:

- (1) **Reward function implementation:** implement and experimentally calibrate the weights β , η , and θ_τ of Equations (2)–(4), currently defined only at a conceptual level;
- (2) **Editing action space implementation:** extend the navigation action space to include the tile flagging and editing actions described in Section 3.4;
- (3) **Data integration:** implement the socket-based communication protocol between Godot and the training environment described in Section 4;
- (4) **Reference level set construction:** prepare the manually annotated level set required for the precision and recall metrics in Section 6;
- (5) **Pilot experiments:** train the first versions of the agent and of the proposed baselines, enabling a first

- still preliminary – empirical validation of the architecture described in this article; and
- (6) **Extension to multiplayer mechanics:** evaluate how the traversal flow $\tau(v)$ should be redefined when multiple agents occupy the same level simultaneously, competing for the same physical space.

10 Conclusion

This article reported on the progress and the architecture under development of a Deep Reinforcement Learning agent for identifying and correcting level-design bottlenecks in 2D games built on the Godot engine. Representing the environment as a 2D matrix, processed by a CNN and consumed by a DRL agent inspired by PCGRL and curiosity-guided exploration, offers a concrete path beyond traditional pathfinding. We formalized the problem as a Markov Decision Process, specified a concrete network architecture (convolutional feature extractor, actor-critic heads, and Intrinsic Curiosity Module) together with a reference set of PPO hyperparameters, and outlined an evaluation methodology – with explicitly defined metrics, baselines, and an experimental protocol – for the next project cycles. We emphasize that, at the time of writing, no training experiments have been run and no performance results exist; every architectural and hyperparameter choice presented here is a proposal to be validated and tuned empirically. We also discussed the most relevant methodological risks, in particular the risk of reward hacking inherent to the use of intrinsic rewards. The most immediate future work – reward function implementation, editing action space, and data integration – will build the necessary foundation for the first pilot training experiments.

Acknowledgements

We thank Prof. Augusto Baffa for his guidance throughout the development of this project, as part of the Projeto Orientado IV (INF1824) course at PUC-Rio.

References

- [1] Pierre Le Pelletier De Woillemont, Rémi Labory, and Vincent Corruble. 2022. Automated Play-Testing Through RL Based Human-Like Play-Styles Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 18.
- [2] Godot Engine Contributors. 2024. Godot Engine – Free and Open Source 2D and 3D Game Engine. <https://godotengine.org>.
- [3] Ahmed Khalifa et al. 2020. PCGRL: Procedural Content Generation via Reinforcement Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 16.
- [4] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-Based Learning Applied to Document Recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.
- [5] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. 2017. Curiosity-Driven Exploration by Self-Supervised Prediction. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [7] Alessandro Sestini et al. 2022. Automated Gameplay Testing and Validation with Curiosity-Conditioned Proximal Trajectories. *IEEE Transactions on Games* 16, 1 (2022), 113–126.
- [8] Noor Shaker, Julian Togelius, and Mark J. Nelson. 2016. *Procedural Content Generation in Games*. Springer.
- [9] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (second ed.). MIT Press.